

Swedish Science Mafia

Experiments, approaches and results

Repository review | Revised 3 October 2026 | Agent addenda incorporated | Updated after all work was merged into main

The strongest completed finding is that executable counterexamples reduce harmful drift in a bounded online bin-packing search. They have not produced an overall improvement over best-fit on the fresh audits. A separate simplification study distinguishes formulas that faithfully reduce to best-fit from poor formulas repaired toward it.

The strategy controller has mixed results: it outperforms several basic fixed strategies, but does not consistently beat a strong patience rule. The LLM triage framework is now on main with a tested integrity gate, but no comparative runs have been saved. The cloud training example remains on a separate branch, also without saved results.

What this report covers

Approach	Evidence status	Pages
Falsify replay and first-fit recovery	Completed synthetic search studies	2
Codex hypothesis and revision pilots	Completed pilot and fresh audit	3
Counterexample promotion gates	V3 strict gates; V4 bounded losses + validation	4-5
Discover, simplify, explain	Preservation vs repair; ablations and alternative rules	6-7
Strategist adaptive research control	Results, hypothesis tests, robustness and forks	8-10
LLM triage, Modal training, smoke test	Infrastructure; no research results saved	11
Interpretation and evidence index	Updated conclusions and source paths	12

Scope and provenance

Reviewed main after pull request #1 merged all Falsify, Strategist, Simplify and Autoresearch work into it, plus the two side branches left out of that merge. This report summarizes recorded evidence and inspects the implementations. Full studies were not rerun. All 45 repository tests pass on a clean export of main (37 standard-library tests and 8 integrity-gate tests under pytest). The simplification report reproduces byte-for-byte, the two-sided simplification counts were recomputed, and reported tables were checked against JSON evidence. Additional historical reproduction claims are attributed to the agent write-ups.

Reviewed: main ff559c5, which includes simplify-explain a0340a8 and claude/zen-faraday-euww4z 781d3b9. Not merged: abdullah-test-modal 6be2a05; abdullah-claude-test d4e3835. Results stored only in external services are outside this assessment.

Counterexample replay and recovery

Question and setting

Can replaying previously observed failures improve search quality at equal evaluator work? Falsify uses online one-dimensional bin packing: bins have capacity 100; each input has 80 integer items; items must be placed in arrival order without future-item access. A fixed evaluator enforces feasible placements. Search mutates 12 weights that score existing feasible bins.

Best-fit is the reference: put an item into the feasible bin leaving the smallest gap. Mean excess bins means candidate bin count minus best-fit bin count, averaged over audit cases and search seeds. Lower is better; zero matches the reference on average.

Three search treatments

Random replay retains random probes. **Counterexample replay** preferentially retains inputs with the greatest historically observed regression. **Counterexample tail** adds a penalty for poor performance on difficult replay inputs. Archives are bounded to 64 inputs; historical failures can become stale as policies change.

Study	Random	Counterexample	Tail penalty
V1: 20 seeds / 120 generations	0.011375	0.006688	0.008313
V2: 50 seeds / 500 generations	0.023000	0.000725	0.000550
First-fit recovery: 20 / 200	0.016000	0.005375	0.004188

Results and differences between versions

V1 used 23,040 search executions per arm per seed and an 800-case audit. Its counterexample-minus-random differences had 95% intervals crossing zero: replay [-0.012000, +0.001938], tail [-0.008688, +0.002313]. It suggested a benefit but did not clearly establish one.

V2 permitted equal-fitness parameter drift and changed the tail objective from absolute bin counts to baseline-relative excess. It used 96,000 search executions per arm per seed and a new 800-case audit across eight families. Replay minus random was -0.022275 bins, 95% interval [-0.037825, -0.010825]; tail minus random was -0.022450, interval [-0.037950, -0.010725]. Both intervals exclude zero.

The recovery study began with first-fit at 0.327500 excess bins. All treatments substantially improved it, and the counterexample treatments ended closer to best-fit: replay minus random was -0.010625 bins, 95% interval [-0.018187, -0.003188]; tail minus random was -0.011813, interval [-0.018812, -0.005000]. Both intervals exclude zero. These were recorded in summary.json but missing from earlier write-ups. None achieved an overall better-than-best-fit audit result.

Conclusion and limits

Replay protects against regression in this search space. The absolute efficiency difference is small. V2 was designed after viewing V1, and its results remain exploratory. Seed bootstrap intervals are conditional on a shared synthetic audit set; they do not measure uncertainty across every possible input distribution. Replay changes the selection data, so it is not an isolated test of explanatory LLM memory.

For comparison, FunSearch (Nature 2024) beat best-fit on OR-Library and Weibull bin packing by evolving code that sees every open bin. Falsify scores each bin separately with fixed features, on 80-item synthetic inputs. Which of these differences prevents gains here is untested.

Evidence: experiments/RESULTS.md; local-v1/summary.json; local-v2/summary.json; local-firstfit/summary.json; PROTOCOL.md; PROTOCOL-v2.md; falsify/README.md (relation to FunSearch).

Codex-guided hypotheses and revisions

This pilot used model-authored hypotheses rather than stochastic mutations alone. Codex proposed a best-fit control and six alternatives, examined measured failures, then proposed six revisions plus a control. The experiment records exact feature vectors and reduced counterexamples. It demonstrates a feedback loop within a conversation, not autonomous API research.

Initial hypotheses: 1,000 exploratory inputs

Candidate	Extra bins	Wins / losses	Idea
best_fit	0.000	0 / 0	Control: smallest feasible gap
avoid_tiny_gaps	0.270	5 / 244	Avoid tiny unusable gaps
avoid_subitem_gaps	0.134	5 / 127	Avoid gaps smaller than the item
tight_or_roomy	0.014	9 / 22	Prefer very tight or roomy gaps
reserve_quarter	0.654	4 / 461	Preserve a gap near 25
reserve_half	0.794	3 / 511	Preserve a gap near 50
avoid_subthird	0.020	8 / 26	Avoid nonzero gaps below one third

A failure that explains the mechanism

For [63, 44, 32, 56], reserve-quarter uses three bins while best-fit uses two. After placing 63 and 44 separately, the remaining spaces are 37 and 56. At item 32, reserve-quarter chooses the second bin, leaving 24. Best-fit chooses the first, leaving 5, and preserves the 56-space bin for the final item. Total item size is 195, so the two-bin packing meets the volume lower bound.

Revisions and independent confirmation

Revisions softened gap penalties, added smooth reciprocal rewards for tight fits, or reduced the preference for roomy bins. The selected moderate tight-or-roomy revision fixed the example and achieved one win, zero losses and 999 ties on the 1,000-case exploratory suite.

Selected revision	Wins	Losses	Ties	Extra bins
Exploratory selection: 1,000	1	0	999	-0.0010
Fresh audit: 10,000 / eight families	3	14	9,983	+0.0011

Conclusion

Fixing a known failure did not generalize into an overall gain. Several conservative revisions tied best-fit during the pilot; an apparent one-case improvement became a net regression in the larger fresh audit. This is a concrete example of why model-generated plausibility and pilot wins require independent confirmation. Reduced failures are not certified globally minimal counterexamples.

Evidence: experiments/codex_candidates.json; codex_revision.json; codex-pilot-v1/results.json; codex-pilot-v2/results.json and audit.json.

Counterexamples as promotion gates

V3 asks whether failures help decide which proposals to deploy, independently of how proposals are generated. Earlier replay studies let the treatment change the search trajectory. Here every arm receives the same four proposals per generation; promotion decisions cannot change future proposals.

Design

40 paired seeds, 400 generations, 89,600 search executions per arm per seed: 10,752,000 across all arms. Policies have 20 features, including eight calculated only from earlier items in the current stream. Candidate generation uses a fixed 24-case suite. Gates inspect 16 sampled random or archived inputs. A new 1,600-case audit covers eight families; a separate 160-case diagnostic suite classifies 960 sampled proposals.

Promotion rule	Extra bins	Mean promotions	Mean rejections
Fixed search score only	0.014000	185.40	0.00
Random-input gate	0.012859	162.20	126.33
Counterexample gate	0.011828	145.75	192.60

Counterexample minus score-only: -0.002171875 bins; paired 95% interval [-0.004109375, -0.000609375]. Counterexample minus random gate: -0.001031250; interval [-0.002390625, +0.00046875]. The first excludes zero; the second does not.

What gets rejected?

Diagnostic class	Sampled proposals	Random rejects	Counterexample rejects
Better on diagnostic suite	62	13 (21.0%)	25 (40.3%)
Worse on diagnostic suite	424	152 (35.8%)	201 (47.4%)
Tied on diagnostic suite	474	34 (7.2%)	52 (11.0%)

These classes reflect a finite diagnostic suite, not statistically established true quality. Samples may include repeated or behaviorally equivalent policies. Counterexample gates are more conservative: they reject more harmful proposals and also more promising ones.

Interpretation

Counterexample gating gives a small benefit over score-only promotion in this setup, but a clear advantage over random gating is unproven. Near-thirds inputs improved in every arm, while regressions elsewhere outweighed those gains. No arm beat best-fit overall. The gate checks sampled archive entries and does not automatically roll back an incumbent after a later probe uncovers a failure.

Execution and verification notes

The saved report records 14 passing Falsify checks and exact seed-0 reproduction. Later review found that archive deduplication could overwrite a larger current-batch regression with a smaller one; prior-generation maxima were retained, but complete within-generation maxima were not. V3 results and source snapshots are preserved. V4 fixes this issue for all its arms. A post-search audit aggregation bug was fixed, and the audit resumed with the original candidates and configuration. The 3,200 discarded audit executions and separate verification rerun are recorded outside the matched search budget. V3 changes several ingredients at once, so comparison with V2 does not isolate the value of contextual features.

Evidence: experiments/gate-v3/RESULTS.md; summary.json; diagnostic.json; execution_note.json; experiments/PROTOCOL-v3.md.

V4: bounded losses and fresh validation

V3 rejected harmful and promising proposals. V4 asks whether limited exceptions, backed by fresh validation, can improve that tradeoff. This completed study was missing from the first report. It uses 40 paired seeds and 200 generations with identical proposals across three equally budgeted arms and the same 20 online features.

Gate rules and shared evaluation

Every candidate is scored on 24 fixed training cases, 16 archived stress cases and 64 fresh validation cases. Each arm requires training performance no worse than its incumbent. All arms pay for the same evaluations, even when they ignore validation. Gates screen proposals and do not automatically roll back incumbents after later failures.

Strict: reject any stress-case loss to best-fit. **Loss budget:** allow losses on at most two stress cases, each costing at most one bin; total positive stress loss must fit one free bin plus any positive net training gain. **Validated budget:** keep those frequency/severity limits, require nonpositive mean validation excess, and use net validation gain to fund any second stress loss. Neutral validation can still permit one isolated one-bin loss.

Independent proposal diagnostic: 960 proposals / 800 cases

Gate	Beneficial allowed / 16	Harmful blocked / 599	Tied rejected / 345
Strict	14 (87.5%)	270 (45.1%)	4
Loss budget	16 (100%)	120 (20.0%)	0
Validated budget	15 (93.8%)	328 (54.8%)	11

Relative to strict, validation retained one more beneficial proposal and blocked 58 more harmful ones. Only 16 proposals were classified as beneficial, so reduced false rejection is not reliably established. The validated gate still allowed 271 empirically harmful proposals. These are finite-suite labels and raw gate decisions, not actual promotions; duplicates or equivalent policies can occur.

Fresh final-policy audit: 1,600 cases / eight families

Gate	Extra bins	Difference from strict	Paired 95% interval
Strict	0.007390625	Reference	-
Loss budget	0.010062500	+0.002671875	[+0.000250, +0.006125]
Validated budget	0.007453125	+0.000062500	[-0.000594, +0.000609]

Plain relaxation worsened final performance. Validation-backed relaxation had no demonstrated final advantage or disadvantage versus strict; an interval crossing zero does not establish equivalence. No arm beat best-fit overall.

Provenance, budget and conclusion

V4 reuses the first 200 generations of the V3 proposal trajectory, so it is not an independent search replication. Its audits are fresh. It also fixes the archive issue, shortens the run and adds validation; differences from V3 cannot be attributed to softer gating alone. Search used 13,056,000 executions, initialization 1,600, final audit 193,600, diagnostics 768,800. No LLM API or GPU was used. Validation plus explicit loss limits is worth further study; weakening rejection alone is not supported.

Evidence: experiments/PROTOCOL-v4.md; soft-gate-v4/RESULTS.md, summary.json, audit.json, diagnostic.json; falsify/soft_gates.py; tests/test_soft_gates.py.

Discover, simplify, explain

Merged from branch simplify-explain (a0340a8). This is a post-search analysis: can an evolved scoring function become a short understandable rule without losing measured quality, and which terms actually matter? It does not alter the original discovery results.

Method

The study combines final V1 policies and recorded Codex proposals into 43 distinct normalized vectors. It uses three seed-separated suites: 500 selection cases, 1,000 simplification cases, and 800 confirmation cases including shifted families. Selection favors the more complex candidate when scores tie.

Weights are normalized by positive scaling. The implementation tests single-term shortcuts, greedily deletes terms, and rounds surviving weights. Changes are accepted if mean bins remain no more than 0.002 above the original on the simplification suite. Each simplification has a 150,000-execution cap. The selected deep dive used 12,000 executions.

The selected winner

The 11-term counterexample_replay-4 policy reduced to a single term: **score = -gap/100**. Maximizing this score means choosing the feasible bin with the smallest remaining gap. The simplified rule is exactly best-fit.

Fresh confirmation: 800 cases	Original	Simplified	Best-fit
Mean bins	40.2850	40.2850	40.2850
Bin-count differences from best-fit	0	0	Reference

Original versus simplified: zero wins, 800 ties, zero losses. Their complete packings were identical on approximately 99.9% of cases. The simplified policy matched best-fit exactly. Removing its only term makes all scores equal, yielding first-fit tie-breaking; this worsened mean bins by 0.3262, with 95% interval [0.2925, 0.3625].

Across the candidate collection

Measure	Result
Distinct candidates	43
Mean number of terms	6.74 before -> 1.05 after
Mean-quality preserved within +/-0.002 bins	17 total; 14 reduce to best-fit
Mean quality improved by more than 0.002 bins	26 total; 21 reduce to best-fit
Total reductions to exactly best-fit	35: 14 preservation + 21 repair
Reduced to first-fit	0

Conclusion and limits

The selected winner reduces faithfully in bin count to best-fit on the confirmation suite. The collection requires a correction: the one-sided acceptance rule can repair a poor candidate as well as explain it. Independently recomputing absolute mean change on the simplification suite gives 17 within +/-0.002 bins and 26 improved by more than that amount. Of the 35 best-fit reductions, only 14 preserve mean quality within tolerance; 21 repair worse candidates. This tolerance concerns mean bin count, not identical placements or universal behavioral equivalence. The eight non-best-fit simplifications were not all independently confirmed. Greedy deletion and rounding do not prove globally minimal explanations or universal equivalence.

Its dependencies are now on main, and report.json reproduces byte-for-byte with the shared falsify/core.py. The alternative rules still need confirmation on fresh data.

Evidence: experiments/SIMPLIFY_PROTOCOL.md; experiments/simplify-v1/report.md, report.json and EXPERIMENT.md.

Simplification: what actually matters

Ablations remove one term at a time from the original 11-term winner and evaluate 800 fresh confirmation cases. Differences are bins after removal minus bins before removal. Confidence intervals use 10,000 paired case resamples. The table is generated from the saved ablation_original JSON.

Original winner: single-term ablations

Removed term	Weight	Extra bins	95% interval	Changed counts	Identical packs
gap/100	-1.000	+0.00375	[0.0000, 0.0088]	3	92.75%
(gap/100)^2	-0.031	+0.00000	[0.0000, 0.0000]	0	100.00%
1/(gap+1)	+0.160	+0.00000	[0.0000, 0.0000]	0	100.00%
0<gap<10	+0.008	+0.00000	[0.0000, 0.0000]	0	100.00%
0<gap<item	+0.061	+0.00000	[0.0000, 0.0000]	0	94.88%
gap>=item	+0.040	+0.00000	[0.0000, 0.0000]	0	100.00%
abs(gap/100-.25)	-0.019	+0.00000	[0.0000, 0.0000]	0	100.00%
abs(gap/100-.5)	+0.152	+0.00000	[0.0000, 0.0000]	0	100.00%
abs(gap/100-.75)	+0.839	+0.00375	[0.0000, 0.0088]	3	96.62%
0<gap<33	+0.181	+0.00000	[0.0000, 0.0000]	0	99.75%
0<gap<50	-0.631	+0.52375	[0.4575, 0.5913]	250	4.25%

Eight of 11 terms change no bin counts when removed. The gap term and distance-from-75 term each affect three inputs. Removing the gap-under-50 penalty worsens mean bins by 0.52375 and changes 250 inputs. This is the strongest load-bearing term in the original formula. Yet the whole formula can be replaced by best-fit without changing any confirmation bin counts: individual ablations reveal interactions, not novelty or universal necessity. The agent addendum said nine zero-effect terms; the JSON shows eight.

Eight simplified policies that are not exactly best-fit

Candidates	Surviving rule / terms
counterexample_replay-2; random_replay-6, -11; counterexample_tail-11	One term: prefer gaps far from 75; best-fit below 75, worst-fit above
random_replay-5, -19	One term: prefer gaps far from 50; best-fit below 50, worst-fit above
counterexample_replay-1	Two terms: best-fit with a roughly 20-unit penalty for nonzero gaps under 33
counterexample_replay-0	Two terms: reciprocal tight-gap reward plus 0.02 reward for room for another copy of the item

These alternatives were assessed on the simplification suite, not independently confirmed. The largest repair examples were reserve_half (42.037 -> 41.245 mean bins), reserve_quarter (41.889 -> 41.245), avoid_tiny_gaps (41.524 -> 41.245), and avoid_subitem_gaps (41.393 -> 41.245). Those are improvements to poor candidates, not evidence their original strategy was preserved.

Evidence: experiments/SIMPLIFY_REVIEW_ADDENDUM.md; experiments/simplify-v1/report.json (ablation_original and map.rows) and EXPERIMENT.md. Winner simplification: 12,000 executions; full run about 23 CPU seconds; the 5 simplification tests pass in the full suite.

Strategist: adaptive research control

Strategist asks when a search should make a small edit, rewrite, crossover with another solution, or restart. It uses stochastic local operators as stand-ins for LLM actions. No LLM calls were made in these experiments.

How the controller works

Within a search line, Thompson sampling chooses an operation using estimated probability of improvement, typical gain and cost. Context includes six stagnation buckets and whether the line holds the best solution. The controller leaves the leader when its expected yield falls below the observed yield of excursions. It resumes the leader when an excursion has stalled as long as the leader had at departure. Discounting allows estimates to change over time.

Confirmatory design

200 seeds per benchmark (1000-1199), with 3,000 proxy cost units per run. Costs are edit 1.2, crossover 1.5, rewrite 2, restart 2; resuming costs zero. Development seeds were used for design and are disclosed in the protocol. These costs represent a model of research effort, not measured token or dollar expenditure.

Controls include edits-only, uniform actions, a static action mix, patience thresholds 16/64/256, a controller without stagnation context, and one without crossover. The timing-shuffled control replays the adaptive run's exact action multiset in random order. Best patience is selected on the confirmatory data itself, deliberately favoring that baseline.

Benchmark and direction	Adaptive	Shuffled	Best patience	Static mix
LABS merit factor (higher)	3.95893	3.96323	4.44562	3.53303
Heilbronn area (higher)	0.010502	0.009241	0.009753	0.010848
NK fitness (higher)	0.733565	0.730375	0.744622	0.718366
Packing train excess (lower)	-0.040833	-0.041042	-0.040625	-0.043958

What the comparisons show

LABS: adaptive beats edits-only, uniform and static mix, but timing-shuffled is indistinguishable and the best patience rule is substantially better. Removing context or crossover also improves the adaptive mean.

Heilbronn: the clearest timing result. Adaptive exceeds shuffled by 0.001261 area units, 95% interval [0.000831, 0.001688]. It also beats edits-only and best patience. Static mix has a slightly higher mean; that difference is inconclusive.

NK: adaptive beats basic fixed strategies, but loses to best patience and both adaptive ablations. Adaptive minus shuffled is +0.003190, interval [-0.000298, +0.006651]. The Holm-adjusted sign test favors adaptive, but the mean-effect interval includes zero: evidence depends on the statistical question.

Evidence: strategist/PROTOCOL.md; strategist/controller.py; experiments/strategist-v1/config.json and summary.json.

Strategist: hypotheses and robustness

Better/worse below means Holm-adjusted exact sign-test $p < 0.05$ over five primary comparisons per benchmark. No clear means that threshold was not met. Each bracket is adaptive seed wins / ties / losses. This compares outcomes across seeds, while the bootstrap interval addresses the mean effect; those questions can differ.

Adaptive versus	LABS	Heilbronn	NK	Packing training
H1: shuffled timing	No clear (98/11/91)	Better (130/0/70)	Better, weak (116/0/84)	No clear (12/175/13)
H2: static mix	Better (150/4/46)	No clear (93/0/107)	Better (139/1/60)	No clear (11/164/25)
H2: uniform	Better (193/2/5)	Better (200/0/0)	Better (200/0/0)	Better (38/159/3)
H2: edits only	Better (152/10/38)	Better (120/0/80)	Better (158/0/42)	No clear (17/165/18)
H3: best patience	Worse (36/12/152)	Better (123/0/77)	Worse (65/0/135)	No clear (18/166/16)

NK timing is weak because its mean-effect 95% interval includes zero, despite the sign-test result. Best patience is chosen post hoc: $T=64$ on LABS and NK; $T=256$ on Heilbronn and packing. This selection favors the baseline. Holm correction does not cover the exploratory ablations or forks.

Ablations: removing complexity

Benchmark	Adaptive	No crossover	No stall context
LABS merit factor	3.9589	4.1623 (+0.2034)	4.1031 (+0.1442)
NK fitness	0.73357	0.74057 (+0.00700)	0.73808 (+0.00452)
Heilbronn area	0.010502	0.010274 (-0.000228)	0.009821 (-0.000681)

Crossover consumes about 27-33% of adaptive moves. Mean restarts per run are 12.6 on LABS, 33.7 on Heilbronn and 11.2 on NK. Removing crossover or context helps LABS/NK but reduces the Heilbronn mean; the no-crossover Heilbronn difference is inconclusive.

Relative gap to the best non-ablation arm

Arm	LABS gap	Heilbronn gap	NK gap	Worst gap
Adaptive	10.9%	3.2%	1.5%	10.9%
Patience 256	4.6%	10.1%	0.4%	10.1%
Patience 64	0%	25.0%	0%	25.0%
Static mix	20.5%	0%	3.5%	20.5%

On these three benchmarks, fixed patience 256 has a similar worst relative shortfall to adaptive. The controller is not uniquely robust. These percentages normalize each benchmark against its best observed non-ablation arm; they are descriptive summaries, not a new significance test.

Development selection inflated LABS performance

The final controller averages 4.292 merit factor on development seeds 0-19, 3.918 on never-used seeds 20-39, and 3.959 on confirmatory seeds. This review reproduced the first two means with current code. The agent reports trying about 15 variants on 20 development seeds: selecting among them likely inflated the development estimate. The frozen confirmatory run exposed this optimism.

Evidence: [output/pdf/STRATEGIST_REVIEW_UPDATES.md](#); [strategist/RESULTS.md](#); [experiments/strategist-v1/summary.json](#); [strategist/PROTOCOL.md](#).

Strategist: causal checks and caveats

Counterfactual forks

At moments when the controller would leave the leader, the search state is copied. Independent forks either follow the controller or stay with small edits, for an equal 300-unit cost window. The saved study uses 40 new seeds (2000-2039), up to six moments per seed, and 20 random continuations per arm per moment.

Benchmark	Moments	P(improve): switch / stay	Mean gain: switch / stay
LABS	238	54.4% / 50.4%	0.4156 / 0.4565
Heilbronn	237	82.1% / 99.8%	0.001000 / 0.002169
NK	236	62.4% / 66.9%	0.03659 / 0.04650

Switching produced lower mean improvement than staying on all three benchmarks over this short horizon. On LABS, it modestly raised the probability of some improvement while reducing its average magnitude. On Heilbronn and NK, both probability and mean gain were lower.

These checks do not show generally beneficial departure moments. Full-run timing and short-window forks answer different questions. Moment-bootstrap intervals ignore shared seeds and may understate uncertainty.

Fork mean-gain differences: follow minus stay

Benchmark	Difference	95% moment-bootstrap interval	Switch / stay better moments
LABS	-0.04095	[-0.07162, -0.01035]	96 / 96
Heilbronn	-0.001169	[-0.001278, -0.001060]	11 / 226
NK	-0.009908	[-0.011613, -0.008266]	46 / 145

Bin packing: training improvement is not generalization

Adaptive averages -0.040833 extra bins versus best-fit in the Strategist report. This is performance on the seed-specific 24-case training suite used to score and select solutions. The bin-packing adapter has no separate fresh audit in this study. Consequently, these negative scores do not establish a generalizable better-than-best-fit algorithm and do not contradict Falsify's held-out results.

Adaptive, timing-shuffled, edits-only and best patience are close on final packing score. Static mix has the best displayed mean among those controls. Much of the final comparison is tied; there is no clear adaptive timing advantage.

Overall interpretation

The paired controls support selective benefits, especially on Heilbronn. Simple patience rules remain competitive or better on LABS/NK; added context and crossover are not consistently helpful.

Documentation and reproducibility status

strategist/README.md contains results and strategist/RESULTS.md supplies the full write-up. After the merge, all 45 repository tests pass on a clean export of main. The prior failure was transient during development. The Strategist agent reports 90 final-code reruns (all 10 arms; three non-packing benchmarks; seeds 1000, 1077, 1199) reproducing logged outcomes bit-identically; this report did not independently repeat those 90 runs. The results use one problem size and one budget per benchmark, with proxy move costs and local operators. Claims about real LLM research efficiency require model-backed runs and measured costs.

Evidence: experiments/strategist-v1/forks.json; strategist/forks.py; strategist/problems.py; strategist/README.md; experiments/HANDOFF.md.

LLM triage and side branches

LLM triage: autoresearch/ (merged into main)

The framework proposes ideas with Claude, ranks them using Jev, and allocates implementation effort by promise: stronger models for favorites, cheaper models for long shots. A fraction of assignments is randomly swapped, and cheap-model improvements can be promoted for stronger-model refinement. These are configured roles in the code, not independently verified claims about current model availability.

The intended same-dollar-budget evaluation compares Jev ranking, random ranking, Claude ranking, all-strong implementations and all-cheap implementations, with three seeds per setting. Proposed outcomes are best score versus spend, ranking AUC, tier success rates, probability calibration and refinement gains.

Four mathematical adapters are included: circle packing, Erdos square packing, Erdos discrepancy, and sum-difference constructions. They have public and hidden instances, construction validators, and stricter checks when a candidate appears to exceed a reference value.

Result status: no committed triage run logs, score curves or comparison summaries were found. The code establishes an implementation and evaluation plan; it does not yet establish useful ranking, cost savings or mathematical discovery. Reference constructions and best-known numbers in the problem README are external baselines, not this project's achieved discoveries.

The integrity gate uses static forbidden-pattern checks and subprocess evaluation. The sandbox source explicitly calls this isolation for an honest search loop, not a security sandbox. The protections should not be presented as a demonstrated hardened evaluator.

Fixed during the merge. Candidate processes inherited the Jev key (TYPESAFE_API_KEY); they now lose it and any variable whose name contains KEY, TOKEN, SECRET or PASSWORD, and a test checks this. No key was exposed, because the loop had never been run. The sum-difference reference now includes the 0.01 size bonus, so the bonus alone cannot trigger a record flag.

Evidence: autoresearch/README.md; triage.py; gate.py; sandbox.py; problems/README.md; tests/test_gate.py (merged from branch 781d3b9).

Cloud ML example: abdullah-test-modal (separate branch, not on main)

The example fine-tunes google/bert_uncased_L-2_H-128_A-2 on Rotten Tomatoes sentiment classification using a Modal T4. Defaults are 4,000 training rows, 1,000 validation rows and three epochs. It computes accuracy and F1, saves the model to a persistent volume, and runs example inference on CPU.

Commit history changes the base model and tokenizer to address compatibility. This demonstrates work on making the pipeline runnable; it is not a saved accuracy experiment. The README's 75-80% accuracy and short runtime/cost estimates are expectations, not observed outcomes in the reviewed files. No saved metrics, logs or trained artifacts were found in the branch tree.

Evidence: Branch 6be2a05: example_ml_pipeline/README.md; train.py; pyproject.toml; model/tokenizer fix commits.

Smoke test: abdullah-claude-test (separate branch, not on main)

Adds a minimal test asserting $\text{add}(2, 3) == 5$. It is a setup check, not research evidence. Both side branches were deliberately left off main and should be distinguished from completed experimental studies.

Conclusions and evidence index

Claims supported by the recorded evidence

- 1. Counterexamples can stabilize bounded search.** V2 gives the strongest replay result: lower held-out regression at matched evaluator budgets. First-fit recovery also favours counterexample replay, with intervals excluding zero. V3 supports a small benefit over score-only promotion, but not a clear advantage over random gates. V4 finds that plain relaxation worsens final performance; fresh-validation relaxation improves the sampled proposal tradeoff but does not demonstrate better final quality.
- 2. Complexity can hide a rediscovered baseline.** The selected simplification winner is exactly best-fit after reduction. Fourteen candidates preserve measured mean quality while reducing to best-fit; another 21 were repaired toward best-fit. These map results have weaker independent confirmation than the selected winner.
- 3. Adaptive control has problem-dependent value.** Heilbronn provides a positive full-run timing result. LABS and NK retain stronger patience baselines; short-horizon forks do not show consistently valuable switches.

Claims not yet established

No overall better-than-best-fit result on the fresh Falsify audits; no causal LLM-memory benefit; no measured Jev triage efficiency or new mathematical discovery; no recorded cloud-model accuracy result; no demonstrated adversarially secure evaluator. The Strategist packing improvements are training scores, not independent audit evidence.

Useful next experiments

For Falsify, retain the strict baseline and test validation-backed bounded losses with a larger supply of beneficial proposals. Softer gates have now been tested; their final-quality benefit remains unproven. Compare no memory, prose evidence and executable evidence under equal model/token budgets. Add OR-Library and Weibull families to compare directly with FunSearch. For simplification, confirm non-best-fit reductions and use two-sided behavior checks when the goal is faithful explanation. For Strategist, test real model operations, measured costs, more budgets, and fresh packing audits. For triage, run its planned paired budget comparisons and save all logs before making efficiency claims.

Evidence index: paths relative to the repository

Study	Primary saved evidence
Replay / recovery	experiments/RESULTS.md; local-v1/, local-v2/, local-firstfit/ summary.json and audit.json
Codex pilots	experiments/codex-pilot-v1/results.json; codex-pilot-v2/results.json and audit.json
V3 and V4 gates	experiments/gate-v3/ and soft-gate-v4/: RESULTS.md, summary.json, audit.json, diagnostic.json; PROTOCOL-v3.md and PROTOCOL-v4.md
Simplification	experiments/simplify-v1/report.md, report.json and EXPERIMENT.md; SIMPLIFY_PROTOCOL.md; SIMPLIFY_REVIEW_ADDENDUM.md
Strategist	strategist/RESULTS.md and PROTOCOL.md; experiments/strategist-v1/: summary.json, forks.json, runs.jsonl, report.html, demo.json; tests/test_strategist.py
LLM triage	autoresearch/README.md; gate.py; sandbox.py; problems/README.md; tests/test_gate.py
Cloud ML (branch only)	abdullah-test-modal: example_ml_pipeline/README.md and train.py

Revision incorporates all three agent addenda, corrects the zero-effect ablation count against JSON, and reflects main after the merge (ff559c5). Repository: github.com/swedishScienceMafia/swedishScienceMafia. This PDF is built by `output/pdf/build_review.py`. External literature and benchmark records were not independently audited for this report.